



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 39

Dr. Asad Mahmood

May 05, 2026

Lecture Outline

- Announcements (Assignment and Quiz 5, Motivational Talk)
- Outline of Previous Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.3 The Basics of Caches
- Outline of Today's Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.3 The Basics of Caches
 - 5.4 Measuring and Improving Cache Performance

Chapter 5

Large and Fast: **Exploiting Memory** **Hierarchy**

Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.6 onwards (self-reading)

Example: Mapping Address to Multiword Cache

- Example: Mapping Address to Multiword Cache Block
- 64 blocks, 16 bytes/block
 - To what block number does *address 1200* map?

Block Size Considerations

- Larger blocks should reduce miss rate due to **better exploitation of spatial locality**
- But in a fixed-sized cache
 - Larger blocks $\Rightarrow$ fewer blocks in the cache $\Rightarrow$ More competition $\Rightarrow$ **increased miss rate**
 - Larger blocks $\Rightarrow$ **pollution**
- **Larger miss penalty**
 - Can override benefit of reduced miss rate
 - Early restart and **critical-word-first** can help

Handling Cache Misses

- On cache hit:
 - Pipelined CPU proceeds normally
- On cache miss:
 1. Stall the CPU pipeline **instead of an exception/interrupt**
 - Some advanced out-of-order processors continue with other instructions
 2. Fetch block from next level of hierarchy
 - What memory address should one use for **Instruction cache miss**
 3. Re-perform the task
 - If **Instruction cache miss**
 - Restart instruction fetch
 - If **Data cache miss**
 - Complete data access

Handling Writes

- On *data-write hit*:
 - Just update the block in cache
 - Can result in the **inconsistency problem!**
 - **Solution 1: Write through**: also update memory
- But writes take longer
 - e.g., if base CPI = 1, 10% of instructions are stores, write to memory takes 100 cycles
 - Effective CPI?
 - **Solution 2: Write buffer**
 - Holds data waiting to be written to memory
 - CPU continues immediately
- Pros/Cons ?

Handling Writes

- **Solution 3: Write-Back:**
 - On data-write hit, just update the block in cache
 - Declare that block 'dirty' and keep track of whether or not a block is dirty
 - When a dirty block is replaced
 - Write it back to memory
 - Can use a write buffer to allow replacing block to be read first
 - Pros/Cons ?

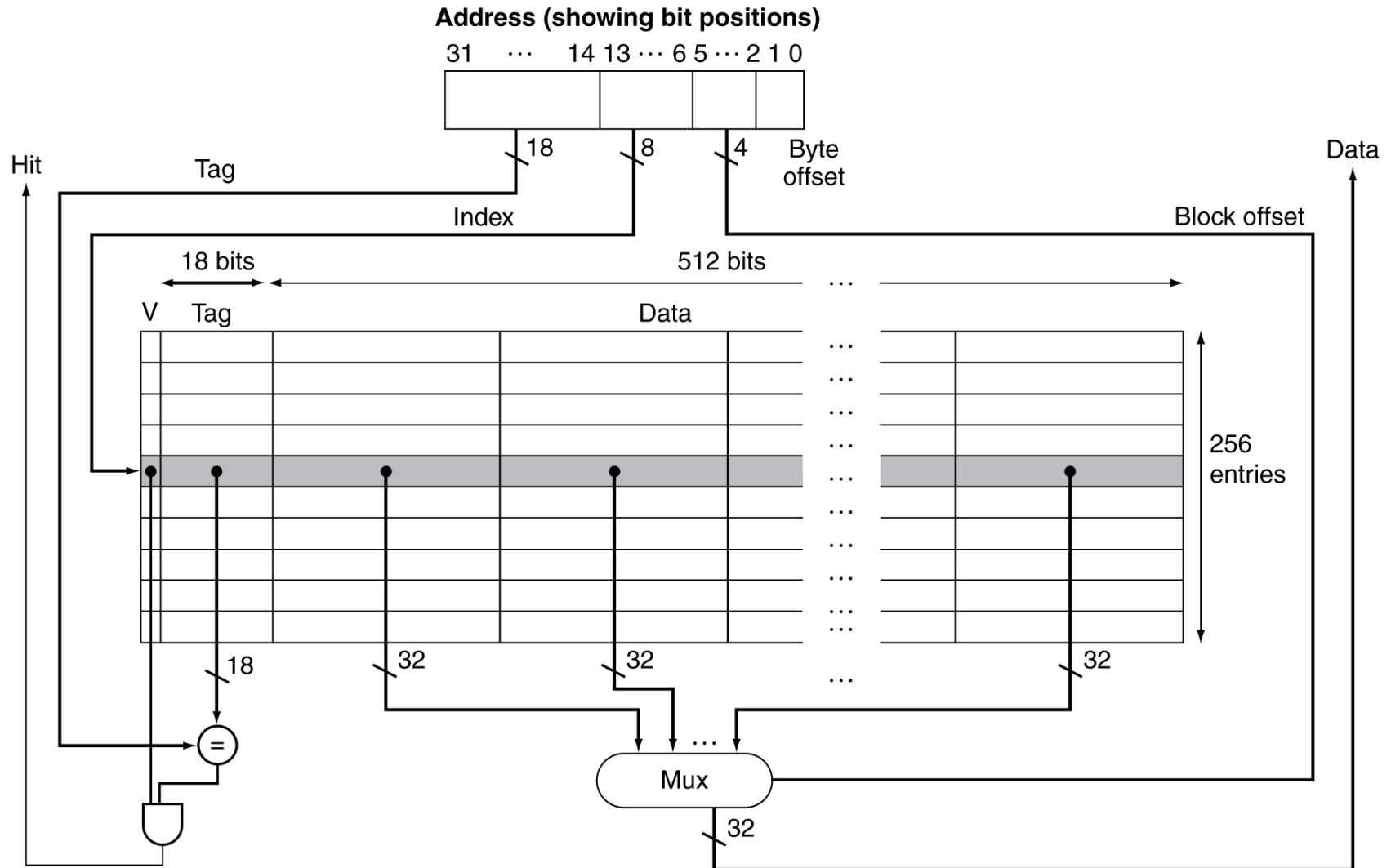
Handling Writes

- What should happen on a *write miss*?
 - **Solution 1: Write Allocate:**
 - *Fetch* the block
 - Perform *write* in cache
 - Usually paired with write-back cache
 - Works best when data is updated frequently
 - **Solution 2: No-Write-Allocate (Write around):**
 - Data written directly into memory
 - Usually paired with write-through policy
 - Works best when data is used once and rarely reused

Example: Intrinsicity FastMATH

- Embedded MIPS processor
 - 12-stage pipeline
 - Instruction and data access on each cycle
- Split cache: separate I-cache and D-cache
 - Each 16KB: 256 blocks × 16 words/block
 - D-cache: write-through or write-back
- SPEC2000 miss rates
 - I-cache: 0.4%
 - D-cache: 11.4%
 - Weighted average: 3.2%

Example: Intrinsic FastMATH



Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.9 onwards (self-reading)

Measuring Cache Performance

- Let us try to quantify cache performance with simplifying assumptions
- Components of CPU time
 1. Program execution cycles
 - Includes cache hit time
 2. Memory stall cycles
 - Mainly from cache misses
 - Read-stall cycles
 - Write-stall cycles (more complicated than read stalls e.g. if write buffer also involved, it can induce stalls as well!)

Cache Performance Example

- Given
 - I-cache miss rate = 2%
 - D-cache miss rate = 4%
 - Miss penalty = 100 cycles
 - Base CPI (ideal cache) = 2
 - Load & stores are 36% of instructions
- Analyze the impact of cache misses on CPU performance vis-à-vis a perfect cache (one that has no misses/stalls)
- What happens if one only improves the CPU performance (e.g. CPI made 1 via improved pipelining) but nothing is done about memory